

A Report Into the Improvement of a Tabletop Gaming Web Application

Exam No. B024703

February 7, 2017

1 Introduction

This report describes a plan of action to improve a web application designed to assist the playing of a table top game. The code was received in a poor state, featuring errors in code that prevent it functioning properly, situations in which poor design decisions were taken by the original author, and situations where the specifications, as provided by the customer, have not yet been fully met.

The web application is written in Python and Uses the Flask web framework.

In this report an approach will be taken that prioritises and improves upon all aspects of these failings in the current approach in an agile iterative fashion, using aspects of the Scrum methodology. The aim is not simply to provide code that is more functional, but that is also extendible and maintainable for further work. Furthermore, as development time will be limited on this project, work will be scheduled so there will be frequent releases of shippable code, as opposed to the waterfall approach which might leave no publishable code when work stops.

2 Issues With The Code

2.1 Compiling and Running the Code

As with the serial case the code was compiled using the Intel C Compiler and using the `-O3` optimisation flag. In this case, however, the appropriate compiler flag was used to compile an OpenMP program. To measure the performance of the different scheduling options the code was initially run on 4 threads on a backend node of `cirrus`, an Intel Xeon powered cluster. For the schedules where it is possible to set a chunksize, this chunksize took the values 1, 2, 4, 8, 32 and 64.

From these results, the scheduling option with the best performance for each loop on 4 threads was determined. Using these optimal options the code was then run again on 1, 2, 4, 6, 8, 12 and 16 threads. This allowed the measurement of the speed up, as defined by T_p/T_1 , so that the scalability of the best performing schedule on 4 threads could be analysed.

For all measurements the running time was measured multiple times and a mean of these values used. In addition the standard deviation was also calculated. This allows the comparison of performance variability between different schedules. Performance variability is often an important consideration in HPC applications, where it may be desirable to avoid methods that often, but unreliably, yield high performance.

3 Results

3.1 Correctness

For almost all HPC applications correctness is the primary concern. The serial code provided included a check sum that could be used to verify the correctness of future parallel results. In all parallel cases this sum yielded the same result as the serial code. We can therefore disregard correctness as a factor when comparing the different schedules.

3.2 Results of Schedules on 4 Threads

The first results gathered were execution times for each loop for every available loop schedule. The results for these are shown in Figures

3.3 Speed Up of Best Schedules

References

- [1] Intel Corp. *OpenMP Loop Scheduling* Accessed: 26-10-2016